

(4) 让用户提前看到一个可工作的系统。

2. RAD 的开发阶段

RAD 的流程从业务建模开始，随后是数据建模、过程建模、应用生成、测试与交付。

(1) 业务建模。确定驱动业务过程运作的信息、要生成的信息、如何生成、信息流的去向及其处理等，可以使用数据流图来帮助建立业务模型。

(2) 数据建模。为支持业务过程的数据流查找数据对象集合、定义数据对象属性，并与其他数据对象的关系构成数据模型，可以使用 E-R 图来帮助建立数据模型。

(3) 过程建模。将数据对象变换为要完成一个业务功能所需的信息流，创建过程以描述增加、修改、删除或获取某个数据对象，即细化数据流图中的加工。

(4) 应用生成。利用第四代语言(4GL)写出处理程序，复用已有构件或创建新的可复用构件，利用环境提供的工具自动生成并构造出整个应用系统。

(5) 测试与交付。因为 RAD 强调复用，许多构件已经是测试过的，这就减少了测试的时间。由于大量复用，所以一般只做总体测试，但新创建的构件还是要测试的。

3. RAD 的特点

RAD 采用基于构件的开发方法，复用已有的程序结构（如果可能的话）或使用构件，或者创建可复用的构件（如果需要的话）。在所有情况下，均可以使用 CASE 工具辅助进行软件构建。如果一个业务能够被模块化，使得其中每一个主要功能均可以在不到三个月的时间内完成，那么，它就是 RAD 的一个候选者。每个主要功能可由一个单独的 RAD 组来实现，最后再集成起来，形成一个整体。

RAD 通过大量使用可复用构件，加快了开发速度。但是，RAD 也具有以下局限性：

(1) 并非所有应用都适合 RAD。RAD 对模块化要求比较高，如果有哪一项功能不能被模块化，那么 RAD 所需要的构件就会有问题；如果高性能是一个指标，且该指标必须通过调整接口使其适应系统构件才能获得，则 RAD 也有可能不能奏效。

(2) 开发者和客户必须在很短的时间完成一系列的需求分析，任何一方配合不当，都会导致 RAD 项目失败。

(3) RAD 只能用于管理信息系统的开发，不适合技术风险很高的情况。例如，当一个新系统要采用很多新技术，或当新系统要与现有系统有较高的互操作性时，就不适合使用 RAD。

7.2.3 统一过程模型

统一过程(Unified Process, UP)是一个通用过程框架，可以用于种类广泛的软件系统、不同的应用领域、不同的组织类型、不同的性能水平和不同的项目规模。UP 是基于构件的，在为软件系统建模时，UP 使用的是 UML。与其他软件过程相比，UP 具有三个显著的特点，即用例驱动、以架构为中心、迭代和增量。

UP 提供了在开发组织中分派任务和责任的纪律化方法，它的目标是在可预见的日程和预算前提下，确保满足最终用户需求的高质量产品。对所有的关键开发活动，它为每个团队成员提供了使用准则、模板和工具指导。而通过对相同基础知识的一致理解，在进行需求分析、设计、

测试或配置管理等工作时，均能确保全体成员共享相同的知识、过程和开发软件的视图。

1. RUP 概述

RUP (Rational Unified Process) 是 Rational 公司开发和维护的过程产品，是由 Objectory 过程演化而来的。RUP 将项目管理、业务建模、分析与设计等统一起来，贯穿整个开发过程。RUP 采用 Internet 技术，可以增强团队的开发效率，并为所有成员提供最佳的软件实现方案，它使团队中每个开发人员的见解和思想得到统一，使开发小组成员的沟通更为容易，而这正是任何项目取得成功的关键因素。RUP 可以增强开发人员对软件的预见性，最终的好处就是提高了软件质量，并有效缩短了软件从开发到投放市场的时间。RUP 为软件开发提供了规范性的指南、模板和范例，可用来开发所有类型的应用。

RUP 中的软件过程在时间上被分解为 4 个顺序的阶段，分别是初始阶段、细化阶段、构建阶段和移交阶段。每个阶段结束时都要安排一次技术评审，以确定这个阶段的目标是否已经满足。如果评审结果令人满意，就允许项目进入下一个阶段。基于 RUP 的软件过程模型如图 7-4 所示。

从图 7-4 中可以看出，基于 RUP 的软件过程是一个迭代过程。初始、细化、构建和移交 4 个阶段就是一个开发周期，每次经过这 4 个阶段就会产生一代软件。除非产品退役，否则，通过重复同样的 4 个阶段，产品将演化为下一代产品，但每一次的侧重点都将放在不同的阶段上。

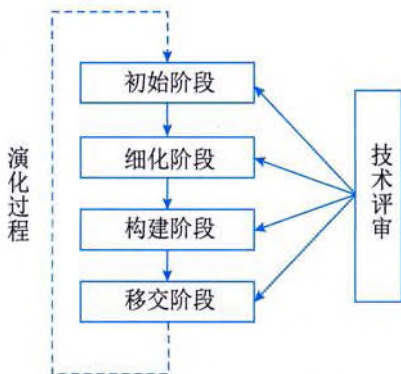


图 7-4 基于 RUP 的软件过程

2. 初始阶段

初始阶段的任务是为系统建立业务模型并确定项目的边界。在初始阶段，必须识别所有与系统交互的外部实体，定义系统与外部实体交互的特性。在这个阶段关注的是整个项目的业务和需求方面的主要风险。对于建立在原有系统基础上的开发项目来说，初始阶段可能很短。初始阶段的实现过程如下：

(1) 明确项目规模。建立项目的软件规模和边界条件，包括验收标准；了解环境及重要的需求和约束，识别系统的关键用例。

(2) 评估项目风险。软件过程主要关心的是软件开发的已知方面，只能准确描述、计划、分配和评审那些已经知道将要完成的事情。风险管理则主要关心未知方面。在基于 RUP 的迭代式软件过程中，很多决策要受风险决定。要达到这个目的，开发人员需要详细了解项目所面临的风险，并对如何降低或处理风险有明确的策略。

(3) 制订项目计划。估计整个项目的总体成本、进度和人员配备。综合考虑备选架构，评估设计和自制/外购/复用方面的方案，从而估算出成本、进度和资源。在这个过程中，要通过对一些概念的证实来证明可行性，可以采用可模拟需求的模型形式或用于探索高风险区的初始原型。初始阶段的原型设计工作应该限制在确信解决方案可行就可以了，具体实现留到细化阶段和构建阶段。

(4) 阶段技术评审。初始阶段结束时要进行一次技术评审，检查初始阶段的目标是否完成，并决定继续进行项目还是取消项目。在评审过程中，需要考虑项目的规模定义、成本和进度估算是否适中、估算根据是否可靠、需求是否正确、开发方和用户方对软件需求的理解是否达成一致、是否已经确定所有风险且有针对每个风险的规避策略等问题。

3. 细化阶段

细化阶段的任务是分析问题领域，建立完善的架构，淘汰项目中最高风险的元素。在细化阶段，必须在理解整个系统的基础上，对架构做出决策，包括其范围、主要功能和诸如性能等非功能需求，同时为项目建立支持环境。细化阶段的实现过程如下：

(1) 确定架构。确保架构、需求和计划足够稳定，充分减少风险，从而能够有预见性地确定开发所需的成本和开发进度。通过处理架构中的关键场景，建立一个已确定基线的架构，并验证其将在适当时间、以合理的成本支持系统需求。

(2) 制订构建阶段计划。为构建阶段制订详细的过程计划，并为其建立基线。

(3) 建立支持环境。包括开发环境、开发流程、支持构建团队所需的工具和自动化/半自动化支持。

(4) 选择构件。评估现有的构件库和潜在构件，充分了解自制/外购/复用决策，以便有把握地确定构建阶段的成本和进度。集成所选构件，并按主要场景进行评估。

(5) 阶段技术评审。评审时，需要检验详细的系统目标和范围、架构的选择，以及主要风险的解决方案。

在细化阶段，可执行的原型依赖于项目的范围、规模、风险和先进程度。必须至少处理初始阶段中识别的关键用例，因为关键用例通常揭示了项目的主要技术风险。

4. 构建阶段

在构建阶段，要开发所有剩余的构件和应用程序功能，把这些构件集成为产品，并进行详细测试。从某种意义上说，构建阶段是一个制造过程，其重点放在管理资源及控制操作，以优化成本、进度和质量。构建阶段的主要任务是通过优化资源和避免不必要的报废和返工，使开发成本降到最低；完成所有所需功能的分析、开发和测试，快速完成可用的版本；确定软件、场地和用户是否已经为部署软件做好准备。

在构建阶段，开发团队的工作可以实现某种程度的并行。一些项目的规模大得足够产生许多并行的增量构建过程，即使是较小的项目，也通常包括可以相互独立开发的构件，从而使各团队之间实现并行开发。这些并行活动在加速版本发布的有效性的同时，也增加了资源管理和工作流同步的复杂性。

构建阶段结束时也要进行技术评审，评审产品是否可以在 β 测试环境中进行安装和运行。

5. 移交阶段

当基线已经足够完善，可以安装到最终用户实际环境中时，则进入移交阶段。移交阶段的重点是确保软件对最终用户是可用的。移交阶段的主要任务是进行 β 测试，制作产品发布版本；对最终用户支持文档定稿；按用户的需求确认新系统；培训用户和维护人员；获得用户对